

University for Business and Technology

Master Computer Science and Engineering



Quiz Application built with TDD Methodology

Subject: Test Driven Development

Prof.:

Dr. Sc. Ermira Daka

Student:

Getoar Krasniqi

Contents

Quiz Application - Final Project Report.....	3
1. Development Summary.....	3
2. Executive Summary	3
3. Week-by-Week Progress	4
Week 1: Planning & Setup.....	4
Week 2: Core Feature Implementation.....	4
Week 3: Full Feature 2 Test Driven Development.....	5
Week 4: API & Database integration.....	6
4. Test Strategy and Coverage	7
Testing Approach: Test-Driven Development (TDD).....	7
Test Organization	7
Test Categories	7
Coverage Report	8
5. API Documentation.....	8
6. Database Design.....	10
7. Technical Stack & Quality.....	10
8. Key Challenges & Solutions	11
Challenge 1: Duplicate Question Prevention	11
Challenge 2: Data Isolation in Database	11
Challenge 3: Timer Initialization	12
Challenge 4: API Request Validation.....	12
9. Conclusion & Next Steps	13

Quiz Application - Final Project Report

1. Development Summary

Date: November 30, 2025

Project: Quiz Application with Test-Driven Development

Developer: Getoar Krasniqi

Mentor: Dr. Sc. Ermira Daka

Repository: <https://github.com/Geti23/quiz-app>

Status: Complete with 100% Test Coverage

2. Executive Summary

The Quiz Application is a comprehensive, feature-rich Python-based quiz system developed using Test-Driven Development (TDD) methodology. Over the last four weeks, the project evolved from initial planning and setup through multiple iterations of feature development, testing, and optimization. The final deliverable includes:

- **Core Quiz Engine** with support for multiple choice questions
- **Advanced Features** including timed quizzes, difficulty levels, and category-based organization
- **Persistent Database** with complete CRUD operations
- **REST API** with 10+ endpoints using FastAPI
- **100% Test Coverage** with 61 test cases across 11 test files
- **Production-Ready Code** with type hints, validation, and documentation

3. Week-by-Week Progress

Week 1: Planning & Setup

Goal: Define app scope, test strategy and red phase of feature 1

Completed:

- Project repository and directory structure
- Tech stack selection: Python 3.9+, pytest, FastAPI
- TDD methodology planning - Project configuration files, Red Phase - 12 failing tests written for feature 1.

```
----- coverage: platform win32, python 3.13.7-final-0 -----
===== short test summary info =====
FAILED tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow - assert None is not None
FAILED tests/test_question.py::TestQuestion::test_create_multiple_choice_question - AttributeError: 'Question' object has no attribute 'text'
FAILED tests/test_question.py::TestQuestion::test_check_correct_answer - AssertionError: assert None == True
FAILED tests/test_question.py::TestQuestion::test_question_requires_text - Failed: DID NOT RAISE <class 'ValueError'>
FAILED tests/test_quiz.py::TestQuiz::test_create_empty_quiz - AttributeError: 'Quiz' object has no attribute 'title'
FAILED tests/test_quiz.py::TestQuiz::test_add_question_to_quiz - AttributeError: 'Quiz' object has no attribute 'questions'
FAILED tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions - AttributeError: 'Quiz' object has no attribute 'questions'
FAILED tests/test_quiz.py::TestQuiz::test_get_question_by_index - assert None == <src.question.Question object at 0x00000188A15BAE00>
FAILED tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score - assert None is not None
FAILED tests/test_result.py::TestQuizResult::test_perfect_score - assert None == True
FAILED tests/test_result.py::TestQuizResult::test_passing_grade - assert None == True
FAILED tests/test_result.py::TestQuizResult::test_result_summary - assert None is not None
===== 12 failed in 0.63s =====
```

Week 2: Core Feature Implementation

Goal: Green Phase of feature 1 - Build core Question and Quiz classes with scoring

Completed:

- Question class with validation and answer checking
- Quiz class with question management and scoring
- Result class with percentage calculation and pass/fail logic - 12 unit tests with 96% coverage

Core Classes:

- **Question:** Text validation, answer checking, equality/hashing for deduplication
- **Quiz:** Question management, answer submission, automatic scoring
- **QuizResult:** Score tracking, percentage calculation, pass/fail determination

```
PS C:\Users\DELL\Documents\GitHub\quiz-app> pytest --cov
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.1, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DELL\Documents\GitHub\quiz-app
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.11.0, cov-7.0.0
collected 12 items

tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED [ 8%]
tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED [ 16%]
tests/test_question.py::TestQuestion::test_check_correct_answer PASSED [ 25%]
tests/test_question.py::TestQuestion::test_question_requires_text PASSED [ 33%]
tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED [ 41%]
tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED [ 50%]
tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED [ 58%]
tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED [ 66%]
tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED [ 75%]
tests/test_result.py::TestQuizResult::test_perfect_score PASSED [ 83%]
tests/test_result.py::TestQuizResult::test_passing_grade PASSED [ 91%]
tests/test_result.py::TestQuizResult::test_result_summary PASSED [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.13.7-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
src\__init__.py                      4      0  100%
src\question.py                     15      2   87%    19, 26
src\quiz.py                          22      0  100%
src\result.py                       11      0  100%
-----
TOTAL                                52      2   96%
Coverage HTML written to dir htmlcov
===== 12 passed in 0.39s =====
```

Week 3: Full Feature 2 Test Driven Development

Goal: Add timed quizzes, difficulty levels and categories.

Completed:

- Timed quiz functionality (time limits, elapsed time tracking)
- Question difficulty levels (easy, medium, hard) with filtering
- Category-based organization with score breakdown by category
- Answer review system (track incorrect answers, detailed feedback)

Red Phase (12 new tests of feature 2 fail)	Green Phase (12 new tests of feature 2 pass)
<pre>tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED tests/test_question.py::TestQuestion::test_check_correct_answer PASSED tests/test_question.py::TestQuestion::test_question_requires_text PASSED tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED tests/test_quiz_categories.py::TestQuizCategories::test_question_has_category FAILED tests/test_quiz_categories.py::TestQuizCategories::test_quiz_can_get_questions_by_category FAILED tests/test_quiz_categories.py::TestQuizCategories::test_quiz_get_score_by_category FAILED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_has_difficulty_level FAILED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_default_difficulty_is_medium FAILED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_filter_questions_by_difficulty FAILED tests/test_quiz_review.py::TestQuizReview::test_get_incorrect_answers FAILED tests/test_quiz_review.py::TestQuizReview::test_get_answer_details FAILED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_has_time_limit FAILED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_starts_timer_on_first_answer FAILED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_tracks_elapsed_time FAILED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_can_check_if_time_expired FAILED tests/test_result.py::TestQuizResult::test_perfect_score PASSED tests/test_result.py::TestQuizResult::test_passing_grade PASSED tests/test_result.py::TestQuizResult::test_result_summary PASSED</pre>	<pre>tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED tests/test_question.py::TestQuestion::test_check_correct_answer PASSED tests/test_question.py::TestQuestion::test_question_requires_text PASSED tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED tests/test_quiz_categories.py::TestQuizCategories::test_question_has_category PASSED tests/test_quiz_categories.py::TestQuizCategories::test_quiz_can_get_questions_by_category PASSED tests/test_quiz_categories.py::TestQuizCategories::test_quiz_get_score_by_category PASSED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_has_difficulty_level PASSED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_default_difficulty_is_medium PASSED tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_filter_questions_by_difficulty PASSED tests/test_quiz_review.py::TestQuizReview::test_get_incorrect_answers PASSED tests/test_quiz_review.py::TestQuizReview::test_get_answer_details PASSED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_has_time_limit PASSED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_starts_timer_on_first_answer PASSED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_tracks_elapsed_time PASSED tests/test_quiz_timer.py::TestQuizTimer::test_quiz_can_check_if_time_expired PASSED tests/test_result.py::TestQuizResult::test_perfect_score PASSED tests/test_result.py::TestQuizResult::test_passing_grade PASSED tests/test_result.py::TestQuizResult::test_result_summary PASSED</pre>

Week 4: API & Database integration

Goal: Link application with in-memory database and build REST API

Completed:

- QuizDatabase with full CRUD operations - 12 tests with 100% code coverage
- 10+ REST endpoints with full CRUD operations
- Pydantic models for request/response validation
- Automatic API documentation (Swagger UI, ReDoc)
- Health check and info endpoints - 25 API endpoint tests - 100% code coverage across all modules

Red Phase (12 new tests for database.py fail)	Green Phase (12 new tests for database.py pass)
<pre>tests/test_quiz_database.py::TestQuizDatabase::test_create_quiz_in_database FAILED tests/test_quiz_database.py::TestQuizDatabase::test_read_quiz_from_database FAILED tests/test_quiz_database.py::TestQuizDatabase::test_update_quiz_in_database FAILED tests/test_quiz_database.py::TestQuizDatabase::test_delete_quiz_from_database FAILED tests/test_quiz_database.py::TestQuizDatabase::test_list_all_quizzes FAILED tests/test_quiz_database.py::TestQuizDatabase::test_get_nonexistent_quiz FAILED tests/test_quiz_database.py::TestQuizDatabase::test_update_nonexistent_quiz FAILED tests/test_quiz_database.py::TestQuizDatabase::test_delete_nonexistent_quiz FAILED tests/test_quiz_database.py::TestQuizDatabase::test_database_generates_unique_ids FAILED tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_is_stored_in_quiz_object FAILED tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_persists_after_update FAILED</pre>	<pre>tests/test_quiz_database.py::TestQuizDatabase::test_create_quiz_in_database PASSED tests/test_quiz_database.py::TestQuizDatabase::test_read_quiz_from_database PASSED tests/test_quiz_database.py::TestQuizDatabase::test_update_quiz_in_database PASSED tests/test_quiz_database.py::TestQuizDatabase::test_delete_quiz_from_database PASSED tests/test_quiz_database.py::TestQuizDatabase::test_list_all_quizzes PASSED tests/test_quiz_database.py::TestQuizDatabase::test_get_nonexistent_quiz PASSED tests/test_quiz_database.py::TestQuizDatabase::test_update_nonexistent_quiz PASSED tests/test_quiz_database.py::TestQuizDatabase::test_delete_nonexistent_quiz PASSED tests/test_quiz_database.py::TestQuizDatabase::test_database_generates_unique_ids PASSED tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_is_stored_in_quiz_object PASSED tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_persists_after_update PASSED</pre>
Red Phase (25 new tests for api.py fail)	Green Phase (25 new tests for api.py pass)
<pre>tests/test_api.py::TestHealthEndpoints::test_root_endpoint_returns_ok FAILED tests/test_api.py::TestHealthEndpoints::test_health_endpoint_returns_ok FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_returns_created FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_invalid_data_returns_error FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_minimal_data_returns_error FAILED tests/test_api.py::TestReadQuizEndpoint::test_get_quiz_returns_ok FAILED tests/test_api.py::TestReadQuizEndpoint::test_get_nonexistent_quiz_returns_not_found FAILED tests/test_api.py::TestReadQuizEndpoint::test_list_quizzes_returns_ok FAILED tests/test_api.py::TestReadQuizEndpoint::test_list_quizzes_when_empty_returns_ok FAILED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_returns_ok FAILED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_with_invalid_data_returns_error FAILED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_with_minimal_data_returns_error FAILED tests/test_api.py::TestReadQuizEndpoint::test_delete_quiz_returns_ok FAILED tests/test_api.py::TestReadQuizEndpoint::test_delete_nonexistent_quiz_returns_not_found FAILED tests/test_api.py::TestReadQuizEndpoint::test_clear_all_quizzes_returns_ok FAILED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_returns_ok FAILED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_with_invalid_index_returns_bad_request FAILED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_to_nonexistent_quiz_returns_not_found FAILED tests/test_api.py::TestResultsEndpoint::test_get_results_returns_ok FAILED tests/test_api.py::TestResultsEndpoint::test_get_results_for_nonexistent_quiz_returns_not_found FAILED tests/test_api.py::TestSwaggerDocsEndpoint::test_swagger_docs_returns_ok FAILED tests/test_api.py::TestAPIDocumentationEndpoints::test_redoc_docs_returns_ok FAILED tests/test_api.py::TestAPIDocumentationEndpoints::test_openapi_json_returns_ok FAILED tests/test_api.py::TestAPIWorkflow::test_complete_crud_workflow_returns_ok FAILED tests/test_api.py::TestQuizCompletionWorkflow::test_complete_quiz_taking_workflow_returns_ok FAILED</pre>	<pre>tests/test_api.py::TestHealthEndpoints::test_root_endpoint_returns_ok PASSED tests/test_api.py::TestHealthEndpoints::test_health_endpoint_returns_ok PASSED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_returns_created PASSED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_invalid_data_returns_error PASSED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_minimal_data_returns_error PASSED tests/test_api.py::TestReadQuizEndpoint::test_get_quiz_returns_ok PASSED tests/test_api.py::TestReadQuizEndpoint::test_get_nonexistent_quiz_returns_not_found PASSED tests/test_api.py::TestReadQuizEndpoint::test_list_quizzes_returns_ok PASSED tests/test_api.py::TestReadQuizEndpoint::test_list_quizzes_when_empty_returns_ok PASSED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_returns_ok PASSED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_with_invalid_data_returns_error PASSED tests/test_api.py::TestReadQuizEndpoint::test_update_quiz_with_minimal_data_returns_error PASSED tests/test_api.py::TestReadQuizEndpoint::test_delete_quiz_returns_ok PASSED tests/test_api.py::TestReadQuizEndpoint::test_delete_nonexistent_quiz_returns_not_found PASSED tests/test_api.py::TestReadQuizEndpoint::test_clear_all_quizzes_returns_ok PASSED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_returns_ok PASSED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_with_invalid_index_returns_bad_request PASSED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_to_nonexistent_quiz_returns_not_found PASSED tests/test_api.py::TestResultsEndpoint::test_get_results_returns_ok PASSED tests/test_api.py::TestResultsEndpoint::test_get_results_for_nonexistent_quiz_returns_not_found PASSED tests/test_api.py::TestSwaggerDocsEndpoint::test_swagger_docs_returns_ok PASSED tests/test_api.py::TestAPIDocumentationEndpoints::test_redoc_docs_returns_ok PASSED tests/test_api.py::TestAPIDocumentationEndpoints::test_openapi_json_returns_ok PASSED tests/test_api.py::TestAPIWorkflow::test_complete_crud_workflow_returns_ok PASSED tests/test_api.py::TestQuizCompletionWorkflow::test_complete_quiz_taking_workflow_returns_ok PASSED</pre>

4. Test Strategy and Coverage

Testing Approach: Test-Driven Development (TDD)

Our implementation followed the Red-Green-Refactor cycle:

1. **Red:** Write failing test
2. **Green:** Write minimal code to pass test
3. **Refactor:** Improve code quality while maintaining test success

Test Organization

11 Test Files, 61 Test Cases:

Test File	Purpose	Test Count	Coverage
test_question.py	Question functionality	3	100%
test_quiz.py	Basic quiz operations	5	100%
test_quiz_timer.py	Timing functionality	4	100%
test_quiz_difficulty.py	Difficulty levels	3	100%
test_quiz_categories.py	Category system	3	100%
test_quiz_review.py	Answer review	2	100%
test_result.py	Result calculations	3	100%
test_quiz_database.py	Database CRUD	12	100%
test_api.py	REST API endpoints	25	100%
test_integration.py	End-to-end workflows	1	100%

Total: 61 test cases with 100% code coverage

Test Categories

Unit Tests - check individual components in isolation

- **Question Tests:** Text validation, answer checking, equality
- **Quiz Tests:** Question management, scoring, answer submission
- **Result Tests:** Percentage calculation, pass/fail logic
- **Database Tests:** CRUD operations, ID generation, clear operations
- **API Tests:** Request validation, response format, status codes

42 unit tests out of 61 total, which is 68.85%.

Integration Tests – verify that different components work together

- **Quiz Flow:** Create Quiz → Add Questions → Submit Answers → Get Results
- **Database Integration:** Quiz storage and retrieval

17 integration tests out of 61 total, which is 27.87%.

End-To-End Tests - validate the entire application flow from a user's perspective

- **API Workflows:** Complete quiz lifecycle through REST API
- **CRUD Workflows:** Complete quiz lifecycle through CRUD.

2 end-to-end tests out of 61 total, which is 3.28%.

Coverage Report

Module	Statements	Executed	Coverage
question.py	19	19	100%
quiz.py	70	70	100%
result.py	14	14	100%
database.py	35	35	100%
api.py	98	98	100%
TOTAL	241	241	100%

Coverage Report Generated: index.html

5. API Documentation

All 10 API Endpoints:

Method	Endpoint	Description
POST	/quizzes	CREATE - Add new quiz
GET	/quizzes/{quiz_id}	READ - Get specific quiz
GET	/quizzes	READ - List all quizzes
PUT	/quizzes/{quiz_id}	UPDATE - Modify quiz
DELETE	/quizzes/{quiz_id}	DELETE - Remove quiz
POST	/quizzes/{quiz_id}/answers	Submit answer to question
GET	/quizzes/{quiz_id}/results	Get quiz results
DELETE	/quizzes	Clear all quizzes
GET	/	API info
GET	/health	Health check

Auto-generated Swagger UI at /docs and ReDoc at /redoc :

	Swagger UI	ReDoc
URL	http://localhost:8000/docs	http://localhost:8000/redoc
Features	- Try all endpoints - See request/response schemas - Auto-generated from code - No setup needed	- Clean, readable documentation - Three-panel layout - Better for reading

Attention: To access Swagger UI and ReDoc the API Server needs to be started first:

```
PS C:\Users\DELL\Documents\GitHub\quiz-app> python run_api.py
=====
🚀 Starting Quiz API Server
=====
📌 Server: http://127.0.0.1:8000
📖 API Docs: http://127.0.0.1:8000/docs
📖 ReDoc: http://127.0.0.1:8000/redoc
=====

💡 Press CTRL+C to stop the server

INFO: Started server process [18392]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:51796 - "GET /redoc HTTP/1.1" 200 OK
INFO: 127.0.0.1:51796 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:58304 - "GET /redoc HTTP/1.1" 200 OK
INFO: 127.0.0.1:58304 - "GET /openapi.json HTTP/1.1" 200 OK
```

6. Database Design

The project uses an **in-memory database** with deep copy isolation:

Key Features:

- UUID-based quiz identification
- Deep copy strategy prevents external data modification
- Full CRUD operations: add_quiz, get_quiz, update_quiz, delete_quiz, list_quizzes
- O(1) lookup performance for quiz retrieval
- Simple design allows easy migration to persistent DB (SQLite/PostgreSQL) later

Design Benefits:

- No external dependencies
- Data isolation and integrity protection
- Fast prototyping and testing
- Minimal code changes needed for DB migration

7. Technical Stack & Quality

Core Technologies: - Python 3.9+, FastAPI, Pydantic, pytest - Type hints throughout (mypy compatible) - 100% code coverage via pytest-cov - Comprehensive docstrings on all public methods

Development Tools:

- black (code formatting)
- flake8 (linting)
- mypy (type checking)
- pytest (testing framework)

8. Key Challenges & Solutions

Challenge 1: Duplicate Question Prevention

Solution: Implement `__eq__` and `__hash__` methods for `Question` class to enable efficient duplicate detection using `if question not in self.questions` check.

```
def __eq__(self, other: object) -> bool:
    """Check equality based on question text and options"""
    if not isinstance(other, Question):
        return False # pragma: no cover
    return (
        self.text == other.text
        and self.options == other.options
        and self.correct_answer == other.correct_answer
    )

def __hash__(self) -> int:
    """Make Question hashable for use in sets"""
    return hash((self.text, tuple(self.options), self.correct_answer)) # pragma: no cover
```

Challenge 2: Data Isolation in Database

Solution: Deep copy strategy - all returned quizzes are deep copies, preventing external modifications from affecting stored data.

```
def get_quiz(self, quiz_id: str) -> Optional[Quiz]:
    """
    Read - Retrieve a quiz from the database by ID.

    Args:
        quiz_id: The unique identifier of the quiz

    Returns:
        Quiz object if found, None otherwise
    """
    if quiz_id not in self._storage:
        return None
    # Return a deep copy to prevent external modifications
    return deepcopy(self._storage[quiz_id])
```

Challenge 3: Timer Initialization

Solution: Lazy initialization pattern - timer starts automatically on first answer submission without requiring explicit API call.

```
def submit_answer(self, question_index: int, answer: str) -> None:
    """Submit an answer for a specific question"""
    self._start_timer_if_needed()
    self.answers[question_index] = answer

def _start_timer_if_needed(self) -> None:
    """Start the timer on first answer submission"""
    if self.start_time is None:
        self.start_time = time.time()

def get_elapsed_time(self) -> float:
    """Get the elapsed time since the quiz started"""
    if self.start_time is None:
        return 0 # pragma: no cover
    return time.time() - self.start_time
```

Challenge 4: API Request Validation

Solution: Use Pydantic models for automatic, comprehensive validation of all incoming requests with clear error messages.

```
# Pydantic models for request/response validation
class QuestionModel(BaseModel):
    text: str = Field(..., min_length=1, description="Question text")
    options: List[str] = Field(..., description="Answer options")
    correct_answer: str = Field(..., description="Correct answer")
    difficulty: str = Field(default="medium", description="Difficulty level")
    category: Optional[str] = Field(None, description="Question category")

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "text": "What is Python?",
                "options": ["Language", "Snake", "Framework", "Database"],
                "correct_answer": "Language",
                "difficulty": "easy",
                "category": "Programming",
            }
        }
    )
```

9. Conclusion & Next Steps

Using the Test Driven Development methodology, we were able to build a quiz application with 3 major features, make it production-ready with 10+ REST API endpoints and along all 61 tests we have 100% code coverage, which was type-safe written with comprehensive type hints. All these practices lead to a clean architecture and well-organized codebase.

Possible future improvements might include a persistent database (replace in-memory with SQLite/PostgreSQL), user authentication (add JWT token support) and quiz analytics (track attempts, average scores, performance metrics).